

TRABAJO FINAL

Carlos Astorga

2026-09-18

Trabajo final

Para aprobar la materia deberán entregar un pdf de Rmarkdown (al igual que los TPs). La consigna es ajustar y comparar al menos 2 modelos computacionales a un conjunto de datos de interés de ustedes (reales o simulados).

Deben:

Describir el conjunto de datos empleado, mostrando las primeras filas de la tabla de datos y comentando qué significa cada variable.

También deben mostrar algunos gráficos que representen lo que quieren ajustar (por ejemplo, proporción de elecciones correctas en una tarea de decisión).

Describir los modelos que van a implementar y por qué los eligieron. Qué hipótesis quieren evaluar al compararlos.

Ajustar los modelos, mostrando el código. Pueden minimizar negative log-likelihood o hacer ajuste bayesiano, lo que prefieran.

Graficar las predicciones del modelo en superposición a las predicciones de los datos empíricos para evaluar cualitativamente el ajuste de los modelos.

Comparar cuantitativamente los modelos con alguno de los criterios utilizados en clase

Discutir cuál modelo es el que mejor ajusta a los datos, si es que hay uno que sea superior de acuerdo a los criterios que vimos en clases.

Si tienen una hipótesis sobre el valor de algún parámetro, testarla.

Se valorará positivamente que incluyan análisis de parameter y model recovery, pero no son obligatorios.

****nicocomay@gmail.com sin problemas.

#INTRODUCCION

Se compararon dos modelos de descuento de esfuerzo, el modelo lineal y el cuadrático utilizando como base de datos “Exp1Data.csv” extraída del artículo “Is all mental effort equal? The role of cognitive demand-type on effort avoidance (Embrey, et al., 2023)”.

#Estructura general de los modelos de descuento del esfuerzo

$$SV = f(R, E; \theta)$$

donde: SV = valor subjetivo R = recompensa E = esfuerzo theta = parámetros individuales

$$P(choice) = \frac{e^{\beta SV_i}}{e^{\beta SV_j} + e^{\beta SV_i}}$$

#Modelo lineal La idea es que cada incremento de esfuerzo produce aproximadamente el mismo decremento en el valor.

$$SV = R - kE$$

Es decir: recompensa — costo lineal del esfuerzo.

#Modelo cuadrático

$$SV = R - kE^2$$

#PLANTEO Voy a comparar dos modelos

Modelo 1 — Linear effort discounting

$$SV = R - kE$$

Modelo 2 — #Modelo cuadrático

$$SV = R - kE^2$$

#CARGA DE BASE DE DATOS

```
datos <- read.csv("Exp1Data.csv")
```

Exploración de base de datos filtrando por el sujeto PID 136057 para explorar como es la dinámica del experimento y preparar la base previo al analisis

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
datos_filtrados <- datos %>% filter(PID == 136057)
str(datos_filtrados)
```

```
## 'data.frame': 599 obs. of 23 variables:
```

```
## $ rt : chr "125821.87499990687" "4236" "null" "3811" ...
```

```
## $ responses : chr "{\"undefined\":\"26\"}" "{\"Q0\":\"Female\"}" NA NA ...
```

```
## $ trial_type : chr "survey-text" "survey-multi-choice" "html-keyboard-response" "html-button"
```

```
## $ trial_index : int 0 1 2 3 4 5 6 7 8 9 ...
```

```
## $ time_elapsed : int 125827 130066 135068 138881 143883 164457 191448 192951 195965 204972 ...
```

```
## $ internal_node_id : chr "0.0-0.0" "0.0-1.0" "0.0-2.0-0.0" "0.0-2.0-1.0" ...
```

```
## $ PID : int 136057 136057 136057 136057 136057 136057 136057 136057 136057 136057 ...
```

```
## $ stimulus : chr NA NA "<span style='font-size: 22px;'> Welcome to the experiment.</br></b>" ...
```

```
## $ key_press : chr NA NA "null" NA ...
```

```
## $ button_pressed : int NA NA NA 0 NA 0 0 NA NA NA ...
## $ phase          : chr NA NA NA NA ...
## $ match          : logi NA NA NA NA NA NA ...
## $ trialNumber    : int NA NA NA NA NA NA NA NA NA NA ...
## $ how_many_back  : int NA NA NA NA NA NA NA NA NA NA ...
## $ currentLetter  : chr NA NA NA NA ...
## $ correct        : logi NA NA NA NA NA NA ...
## $ percentCorrect : num NA NA NA NA NA NA NA NA NA NA ...
## $ comparison     : chr NA NA NA NA ...
## $ cogED_Choice   : chr NA NA NA NA ...
## $ choiceNumber   : int NA NA NA NA NA NA NA NA NA NA ...
## $ indifferencePoint: num NA NA NA NA NA NA NA NA NA NA ...
## $ preference     : chr NA NA NA NA ...
## $ finalPay       : num NA NA NA NA NA NA NA NA NA NA ...
```

Se observan en cada sujeto 9 comparaciones (comparison) en donde para cada una hay 7 elecciones (cogED_Choice) a partir del cual se obtiene un punto de indiferencia (Indifference point) y una categoría de dificultad preferida, fácil o difícil (Preference)

```
library(stringr)

# Seleccionar solamente las elecciones

elecciones <- datos %>%
  filter(!is.na(comparison)) %>%
  mutate(

    # Número de dificultad de cada opción
    E_easy = as.numeric(str_extract(comparison, "^\\d+")),
    E_hard = as.numeric(str_extract(comparison, "(?<=v )\\d+")),

    # Recompensas mostradas en cogED_Choice
    reward = as.numeric(str_extract(cogED_Choice, "\\d+\\.\\d+")),

    # Opción elegida
    # 0 = izquierda / fácil
    # 1 = derecha / difícil
    choice = button_pressed
  )

# Reconstruir la recompensa de cada opción

# Reconstruir las recompensas de las dos opciones

elecciones <- elecciones %>%
  group_by(PID, comparison) %>%
  group_modify(~ {

    d <- .x

    leftMoney <- 2
    rightMoney <- 2
    additive <- 1
```

```

titrateSide <- 2

d$R_left <- NA
d$R_right <- NA

for(i in 1:nrow(d)){

  # Guardar las recompensas mostradas
  d$R_left[i] <- leftMoney
  d$R_right[i] <- rightMoney

  choice <- d$choice[i]

  if(i == 1){

    if(choice == 0){
      titrateSide <- 0
      leftMoney <- leftMoney - additive
    }

    if(choice == 1){
      titrateSide <- 1
      rightMoney <- rightMoney - additive
    }

  } else {

    if(titrateSide == 0){

      if(choice == 0){
        leftMoney <- leftMoney - additive
      }

      if(choice == 1){
        leftMoney <- leftMoney + additive
      }
    }

    if(titrateSide == 1){

      if(choice == 0){
        rightMoney <- rightMoney + additive
      }

      if(choice == 1){
        rightMoney <- rightMoney - additive
      }
    }
  }

  additive <- additive / 2
}

```

```

    d
  }) %>%
ungroup()

# Pasar izquierda/derecha a fácil/difícil

elecciones <- elecciones %>%
  mutate(
    R_easy = R_left,
    R_hard = R_right
  )

# Crear la tabla para el modelo

base_modelo <- elecciones %>%
  select(
    PID,
    comparison,
    choiceNumber,
    E_easy,
    E_hard,
    R_easy,
    R_hard,
    choice,
    indifferencePoint,
    preference
  )

# Comprobar valores faltantes

colSums(is.na(base_modelo))

```

```

##          PID          comparison      choiceNumber      E_easy
##          0              0              0              0
##      E_hard      R_easy      R_hard      choice
##          0              0              0              0
## indifferencePoint      preference
##          2106          2106

```

#Ajustando los dos modelos a los datos empíricos Pasos: 1. Definir una función que compute el negative log-likelihood para cada modelo. 2. Ajustar cada modelo con optim(). 3. Guardar los parámetros optimizados y el valor obtenido de log-likelihood

```

####----- paso 1: funciones que computan el negloglik -----
negll_ML <- function(parameters, data){

k <- parameters[1]      # costo del esfuerzo
beta <- parameters[2]   # sensibilidad a las diferencias de valor

# Restricciones de parámetros

```

```

if (k < 0 | beta <= 0) {
  return(1e20)
}

n <- nrow(data)
ll <- rep(NA, n)

# Loop a lo largo de los trials
for (trial in 1:n) {

  # Valor subjetivo de cada opción
  SV_easy <- data$R_easy[trial] - k * data$E_easy[trial]
  SV_hard <- data$R_hard[trial] - k * data$E_hard[trial]

  # Softmax
  p <- exp(beta * c(SV_easy, SV_hard))
  p <- p / sum(p)

  # Elección del participante
  choice <- data$choice[trial]

  # Log-likelihood
  ll[trial] <- ifelse(
    choice == 1,
    log(p[1]),
    log(p[2])
  )
}

# Negative log-likelihood
negll <- -sum(ll)

return(negll)
}

negll_MC <- function(parameters, data){

  k <- parameters[1]      # costo del esfuerzo
  beta <- parameters[2]   # sensibilidad a las diferencias de valor

  # Restricciones de parámetros
  if (k < 0 | beta <= 0) {
    return(1e20)
  }

  n <- nrow(data)
  ll <- rep(NA, n)

  # Loop a lo largo de los trials
  for (trial in 1:n) {

    # Valor subjetivo de cada opción

```

```

SV_easy <- data$R_easy[trial] - k * data$E_easy[trial]^2
SV_hard <- data$R_hard[trial] - k * data$E_hard[trial]^2

# Softmax
p <- exp(beta * c(SV_easy, SV_hard))
p <- p / sum(p)

# Elección del participante
choice <- data$choice[trial]

# Log-likelihood
ll[trial] <- ifelse(
  choice == 1,
  log(p[1]),
  log(p[2])
)
}

# Negative log-likelihood
negll <- -sum(ll)

return(negll)
}

####----- paso 2: ajustar con optim -----
####----- paso 2: ajustar los modelos con optim -----

n_subj <- unique(base_modelo$PID)

fit_ML <- data.frame()
fit_MC <- data.frame()

for(i in n_subj){

  subjdata <- base_modelo[base_modelo$PID == i, ]

  #-----
  # MODELO LINEAL
  #-----

  init_vec <- c(
    runif(1),          # k
    runif(1, 0.5, 15) # beta
  )

  estimate <- optim(
    init_vec,
    fn = negll_ML,
    data = subjdata
  )

  # Repetir con distintos valores iniciales

```

```

for(j in 1:5){

  init_vec <- c(
    runif(1),          # k
    runif(1, 0.5, 15) # beta
  )

  new_est <- optim(
    init_vec,
    fn = negll_ML,
    data = subjdata
  )

  if(new_est$value < estimate$value){
    estimate <- new_est
  }
}

# Guardar resultados
fit_ML <- rbind(
  fit_ML,
  data.frame(
    PID = i,
    k = estimate$par[1],
    beta = estimate$par[2],
    loglik = -estimate$value
  )
)

#-----
# MODELO CUADRÁTICO
#-----

init_vec <- c(
  runif(1),          # k
  runif(1, 0.5, 15) # beta
)

estimate <- optim(
  init_vec,
  fn = negll_MC,
  data = subjdata
)

# Repetir con distintos valores iniciales
for(j in 1:5){

  init_vec <- c(
    runif(1),          # k
    runif(1, 0.5, 15) # beta
  )

```

```

new_est <- optim(
  init_vec,
  fn = negll_MC,
  data = subjdata
)

if(new_est$value < estimate$value){
  estimate <- new_est
}
}

# Guardar resultados
fit_MC <- rbind(
  fit_MC,
  data.frame(
    PID = i,
    k = estimate$par[1],
    beta = estimate$par[2],
    loglik = -estimate$value
  )
)
}

```

#Comparación cuantitativa con Bayesian Information Criterion

```

# Comparación cuantitativa con BIC

# Cantidad de parámetros
k_ML <- 2 # k y beta
k_MC <- 2 # k y beta

# Cantidad de observaciones por participante
n <- nrow(base_modelo)

# BIC para cada participante
fit_ML$bic <- -2 * fit_ML$loglik + k_ML * log(n)
fit_MC$bic <- -2 * fit_MC$loglik + k_MC * log(n)

# Comparación de los modelos
comp <- data.frame(
  loglik = c(
    sum(fit_ML$loglik),
    sum(fit_MC$loglik)
  ),

  bic = c(
    sum(fit_ML$bic),
    sum(fit_MC$bic)
  ),

  modelo = c(
    "Modelo lineal",
    "Modelo cuadrático"
  )
)

```

```
)
)

knitr::kable(
  comp,
  caption = "Comparación cuantitativa de los modelos"
)
```

Table 1: Comparación cuantitativa de los modelos

loglik	bic	modelo
-1606.191	3821.305	Modelo lineal
-1614.029	3836.981	Modelo cuadrático

Conclusión De acuerdo a los valores de BIC se reporta que el modelo lineal cuenta con mejor ajuste con los datos.

#----- #Model recovery

¿Cómo podemos estar seguros de que el diseño usado nos permite distinguir entre los modelos testeados? Para ello hay que hacer un análisis de recuperación de modelos, o model recovery. Como vimos en clase, implica: 1. Simular datos con un modelo. 2. Ajustar esos datos con todos los modelos. 3. Comparar esos modelos con el criterio de comparación elegido (BIC en nuestro caso). 4. Contar la proporción de veces que el modelo ganador es el que simuló los datos. 5. Repetir simulando con todos los modelos. Un buen model recovery implica que cuando simulamos datos con un modelo, el mismo modelo es seleccionado como ganador la mayoría del tiempo.

```
####--- función para simular datos ----
library(ggplot2)
sim_data <- function(data, modelo, k, beta) {

  # Copio las condiciones experimentales reales
  sim <- data

  # Creo la variable de elección
  sim$choice <- NA

  # Simulo cada elección
  for(i in 1:nrow(sim)) {

    # Modelo lineal
    if(modelo == "lineal") {

      SV_easy <- sim$R_easy[i] - k * sim$E_easy[i]
      SV_hard <- sim$R_hard[i] - k * sim$E_hard[i]

    }

    # Modelo cuadrático
    if(modelo == "cuadratico") {

      SV_easy <- sim$R_easy[i] - k * sim$E_easy[i]^2
      SV_hard <- sim$R_hard[i] - k * sim$E_hard[i]^2

    }

  }

}
```

```

}

# Softmax
p <- exp(beta * c(SV_easy, SV_hard))
p <- p / sum(p)

# Simulo la elección
# 0 = easy
# 1 = hard

sim$choice[i] <- sample(
  c(0, 1),
  size = 1,
  prob = p
)
}

return(sim)
}

####--- paso 1: simulamos datos con los dos modelos ----

sim_ML <- data.frame()
sim_MC <- data.frame()

for(i in 1:30){

  ### sim Modelo lineal

  k <- runif(1)
  beta <- runif(1, 0.5, 15)

  temp <- sim_data(
    base_modelo,
    modelo = "lineal",
    k = k,
    beta = beta
  )

  temp$participante <- i

  # Guardamos los parámetros generativos
  # para hacer también parameter recovery

  temp$true_k <- k
  temp$true_beta <- beta

  # Guardo

  sim_ML <- rbind(sim_ML, temp)
}

```

```

### sim Modelo cuadrático

k <- runif(1)
beta <- runif(1, 0.5, 15)

temp <- sim_data(
  base_modelo,
  modelo = "cuadrático",
  k = k,
  beta = beta
)

temp$participante <- i + 30

# Guardamos los parámetros generativos

temp$true_k <- k
temp$true_beta <- beta

# Guardo

sim_MC <- rbind(sim_MC, temp)
}

# Junto datos

sim_ML$true_model <- "Modelo lineal"
sim_MC$true_model <- "Modelo cuadrático"

datos_simulados <- rbind(
  sim_ML,
  sim_MC
)

####---- paso 2: ajustar los datos con los dos modelos ----

fit_ML <- data.frame()
fit_MC <- data.frame()

for(i in 1:length(unique(datos_simulados$participante))){

  subjdata <- datos_simulados[
    datos_simulados$participante == i,
  ]

  ### Modelo lineal

  init_vec <- c(
    runif(1),      # k

```

```

  runif(1, 0.5, 15) # beta
)

estimate <- optim(
  init_vec,
  fn = negll_ML,
  data = subjdata
)

for(j in 1:5){

  init_vec <- c(
    runif(1),          # k
    runif(1, 0.5, 15) # beta
  )

  new_est <- optim(
    init_vec,
    fn = negll_ML,
    data = subjdata
  )

  if(new_est$value < estimate$value){
    estimate <- new_est
  }
}

# Paso 3: guardar

fit_ML <- rbind(
  fit_ML,
  data.frame(
    k = estimate$par[1],
    beta = estimate$par[2],
    loglik = -estimate$value,
    participant = i
  )
)

### Modelo cuadrático

init_vec <- c(
  runif(1),          # k
  runif(1, 0.5, 15) # beta
)

estimate <- optim(
  init_vec,
  fn = negll_MC,
  data = subjdata
)

```

```

for(j in 1:1){

  init_vec <- c(
    runif(1),          # k
    runif(1, 0.5, 15) # beta
  )

  new_est <- optim(
    init_vec,
    fn = negll_MC,
    data = subjdata
  )

  if(new_est$value < estimate$value){
    estimate <- new_est
  }
}

fit_MC <- rbind(
  fit_MC,
  data.frame(
    k = estimate$par[1],
    beta = estimate$par[2],
    loglik = -estimate$value,
    participant = i
  )
)
}

####---- paso 3: comparar modelos ----

# BIC Modelo lineal

fit_ML$bic <- -2 * fit_ML$loglik +
  2 * log(63)

# BIC Modelo cuadrático

fit_MC$bic <- -2 * fit_MC$loglik +
  2 * log(63)

# ¿Cuál es el mejor?

best <- ifelse(
  fit_ML$bic < fit_MC$bic,
  "Modelo lineal",
  "Modelo cuadrático"
)

```

####---- paso 4: graficar matriz de model recovery ----

```
model_rec <- data.frame(  
  part_sim = 1:60,  
  true_model = c(  
    rep("Modelo lineal", 30),  
    rep("Modelo cuadrático", 30)  
  ),  
  best_model = best  
)  
  
recovery_prop <- prop.table(  
  table(  
    model_rec$true_model,  
    model_rec$best_model  
  ),  
  margin = 1  
)  
  
as.data.frame(recovery_prop) %>%  
  ggplot(  
    aes(  
      x = Var2,  
      y = Var1,  
      fill = Freq  
    )  
  ) +  
  geom_tile(show.legend = FALSE) +  
  geom_text(  
    aes(label = round(Freq, 2)),  
    size = 10,  
    fontface = "bold"  
  ) +  
  labs(  
    x = "Modelo seleccionado",  
    y = "Modelo generativo",  
    fill = "Proporción"  
  ) +  
  scale_fill_gradient(  
    low = "yellow",  
    high = "blue"  
  ) +  
  theme_classic(base_size = 14)
```

